

WurieAI API and Secret Setup Guide

This guide covers the credentials and configuration needed to run WurieAI locally and deploy it safely with GitHub Actions, Firebase, Gemini, LangSmith, and Cloud Run.

Security rule

Never paste a real secret into Git, ``.env.example``, the Android project, an APK, a pull request, or chat. Use placeholders in templates and secret stores for real values. If a key has already been shared, revoke it and create a replacement.

Architecture

- Android app: Firebase Authentication, Firebase Android configuration, and calls to the WurieAI backend.
- Firebase: Authentication, Firestore, and the Android app registration.
- Cloud Run: hosts the FastAPI backend and LangGraph agents.
- Gemini: server-side model provider called by the backend agents.
- LangSmith: tracing, evaluation, and agent observability.
- GitHub Actions: builds versioned APKs and deploys the backend.

1. Firebase project

1. Open <https://console.firebase.google.com/>.
2. Create or select the WurieAI Firebase project.
3. Enable Authentication providers required by the app, at minimum Email/Password and Google if Google sign-in is enabled.
4. Create a Firestore database in production mode.
5. Add the Android app using the exact application ID from ``app/build.gradle.kts``.
6. Download ``google-services.json`` and place it at ``app/google-services.json``.
7. Configure Firestore security rules so users can access only their own profile/settings and permitted social data. Admin/provider approval must be enforced server-side.

The Android ``google-services.json`` contains public client configuration. It is not a replacement for Firebase Admin credentials and should not be used as the backend service account.

2. Firebase Admin service account

The backend verifies Firebase ID tokens and accesses Firestore with Firebase Admin.

1. Open <https://console.cloud.google.com/>.
2. Select the same Firebase/GCP project.
3. Go to IAM & Admin -> Service Accounts.
4. Create a dedicated runtime service account, for example ``wuriei-backend-runtime``.
5. Grant only the roles required for Firebase Admin and Firestore access.
6. For local development, create a JSON key only when necessary and store it outside the repository.
7. Set ``FIREBASE_SERVICE_ACCOUNT_PATH`` locally, or store the complete JSON in Secret Manager as ``FIREBASE_SERVICE_ACCOUNT_JSON`` for Cloud Run.

Never commit the JSON service-account file.

3. Gemini API

1. Open <https://aistudio.google.com/app/apikey>.
2. Create a Gemini API key for the backend project.
3. Restrict the key where possible and monitor quotas.
4. Store it locally as ``GEMINI_API_KEY`` in ``wurie-backend/.env``.
5. Store it in Google Secret Manager under ``GEMINI_API_KEY`` for Cloud Run.

The Gemini key must not be in the root Android ``.env``, Android ``BuildConfig``, Firebase client configuration, or APK. The mobile app calls the backend; the backend calls Gemini.

4. LangSmith tracing

1. Open <https://smith.langchain.com/>.
2. Create or select a workspace and tracing project, such as ``wuriei-production``.
3. Create a LangSmith API key.
4. Revoke any key previously shared publicly.
5. Store the replacement locally as ``LANGSMITH_API_KEY`` in ``wurie-backend/.env``.
6. Store it in Google Secret Manager under ``LANGSMITH_API_KEY`` for Cloud Run.
7. Set ``LANGSMITH_TRACING=true`` and ``LANGSMITH_PROJECT=wuriei-production`` in Cloud Run.
8. Use the workspace skill at ``.github/skills/langsmith-trace/SKILL.md`` for trace queries and debugging.

LangSmith is for agent tracing, evaluation, and supported agent deployment workflows. It does not replace the FastAPI host for the profile, provider, booking, wallet, and social APIs.

5. Local backend environment

Copy the template:

```
cp wurie-backend/.env.example wurie-backend/.env
```

Fill in real values only on your machine. For local Firebase credentials, use:

```
FIREBASE_SERVICE_ACCOUNT_PATH=/secure/path/firebase-service-account.json
GEMINI_API_KEY=...
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=...
LANGSMITH_PROJECT=wurieai-development
WURIE_DEV_AUTH_FALLBACK=false
```

The current Python service reads Firebase and tracing environment variables. Run the backend from `wurie-backend` with uv or the repository virtual environment.

6. GitHub Actions secrets and variables

Create these repository secrets under Settings -> Secrets and variables -> Actions:

- `GCP\_WORKLOAD\_IDENTITY\_PROVIDER`: full Workload Identity Federation provider resource name.
- `GCP\_DEPLOY\_SERVICE\_ACCOUNT`: GitHub deployer service-account email.
- `GCP\_RUNTIME\_SERVICE\_ACCOUNT`: Cloud Run runtime service-account email.

Create this repository variable:

- `GCP\_PROJECT\_ID`: Google Cloud project ID.

The workflow also uses the automatically provided `GITHUB\_TOKEN` to create releases.

Do not put Gemini, Firebase Admin, or LangSmith values in GitHub workflow YAML. The deployment workflow reads them from Google Secret Manager at runtime.

7. Google Cloud setup for Cloud Run

1. Select the Firebase project in Google Cloud Console.
2. Enable Cloud Run, Cloud Build, Artifact Registry, Secret Manager, IAM Credentials, and Service Usage APIs.
3. Create the deployer and runtime service accounts.
4. Configure GitHub Actions Workload Identity Federation so GitHub can deploy without a long-lived JSON key.
5. Create Secret Manager secrets named:
 - `GEMINI\_API\_KEY`
 - `FIREBASE\_SERVICE\_ACCOUNT\_JSON`
 - `LANGSMITH\_API\_KEY`
6. Grant the Cloud Run runtime service account Secret Manager Secret Accessor on those secrets.
7. Grant the GitHub deployer permission to deploy Cloud Run services and build source deployments.
8. Push a change under `wurie-backend/` to trigger `.github/workflows/deploy-backend.yml`.

The service is deployed as `wurieai-backend` in `us-central1` by default.

8. Android build and APK releases

The root `.env.example` is intentionally limited to the public Google OAuth Web Client ID used by Google Sign-In. Do not add server secrets there.

The APK workflow supports:

- Normal main-branch build: `0.1.<GitHub run number>`.
- Semantic release tag: `v1.5.0` becomes version name `1.5.0`.
- Version code: `100000 + GitHub run number`.

Create a versioned APK release with:

```
git tag v1.5.0
git push origin v1.5.0
```

The workflow uploads an artifact and creates a GitHub release with a filename similar to:

```
wurieAI-debug-1.5.0-100042.apk
```

This is a debug APK. A Play Store release requires a protected upload keystore in GitHub Secrets and a signed release AAB/APK workflow.

9. Rotation and incident response

If any credential is exposed:

1. Revoke it immediately at the issuing provider.
2. Create a replacement.
3. Update Secret Manager or the local ignored `.env`.
4. Redeploy Cloud Run.
5. Check GitHub Actions logs and LangSmith/GCP audit logs.
6. Do not try to hide the old key by editing history alone; it must be revoked.

Final checklist

- [] Firebase Authentication providers enabled.
- [] Firestore rules reviewed.
- [] `google-services.json` matches the Android application ID.
- [] Firebase Admin service account stored outside Git.
- [] Gemini key stored only in backend secret storage.

- [] LangSmith key rotated and stored only in backend secret storage.
- [] Cloud Run runtime can read all required secrets.
- [] GitHub Workload Identity Federation configured.
- [] GitHub repository variables/secrets configured.
- [] Backend deployment workflow succeeds.
- [] APK release tag produces the expected versioned artifact.